Conjur OSS Lab — Governance Documentation

Security Controls Matrix, Operational Runbooks, Key Custody, Break-Glass Procedure, and Production Considerations

Companion to the Technical Build Record. Structured around the Discover, Document, Tier, Remediate, Monitor methodology.

Contents

Architecture Diagram

1. Security Controls Matrix

1.1 Access control and identity
1.2 Secrets management and lifecycle
1.3 Network and transport security
1.4 Audit and monitoring
1.5 Container and host hardening

2. Application Onboarding Runbook

3. Credential Rotation Runbook

4. Key Custody Procedure

5. Break-Glass Procedure

6. Vulnerability and Benchmark Scan Methodology

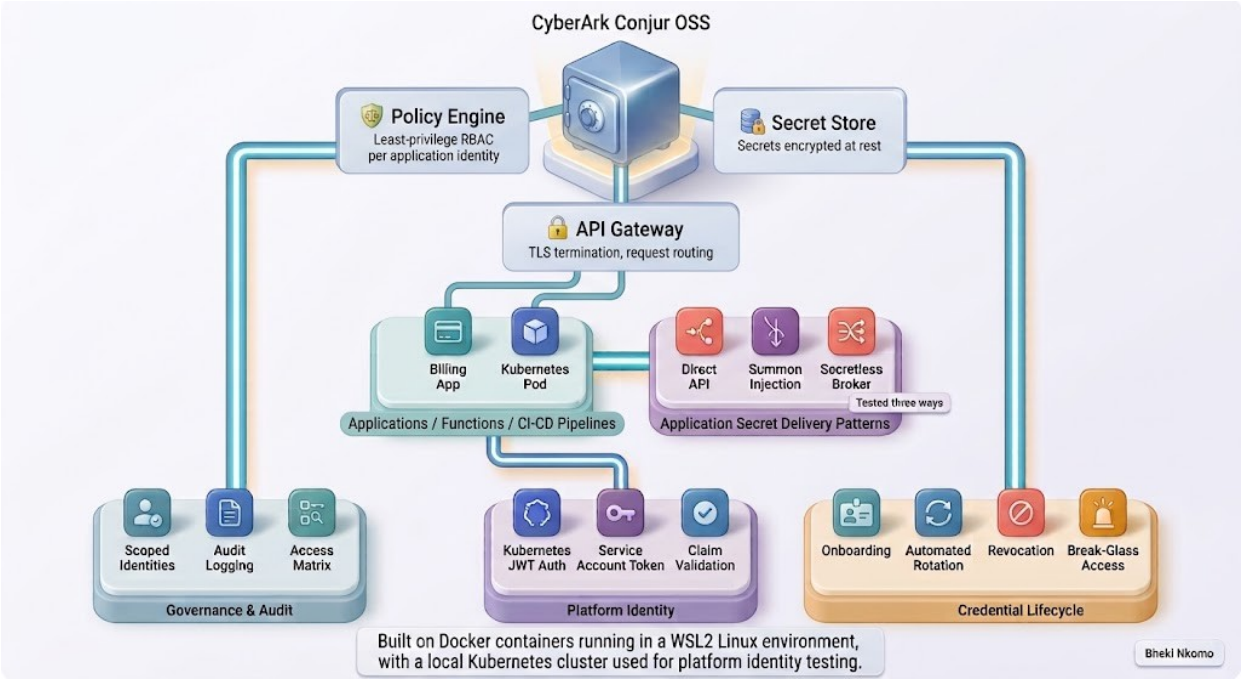
7. Secret Inventory

8. Access Matrix

9. Consolidated Production Considerations

Architecture Diagram

Organised around vault function rather than infrastructure: core functions at top, delivery patterns and platform identity in the middle, governance and lifecycle operations at the bottom. Only the Billing App was exercised through all three delivery patterns; Operations App and Procurement App have vault identities but were not delivery-tested, and are left off accordingly. Full container-level infrastructure detail is in the Technical Build Record, Section 1.



1. Security Controls Matrix

This matrix maps each control implemented or documented in the lab against the applicable standard. Controls are categorised as Implemented (working in the lab), Documented (design exists, not technically enforceable in this environment), or Out of Scope (requires infrastructure not present). The three reference standards are CIS Docker Benchmark v1.6, NIST SP 800-190 (Application Container Security Guide), and ISO 27001:2022 Annex A.

1.1 Access control and identity

Control	Standard ref	Status	Implementation / gap
Each application authenticates as a distinct, named machine identity	ISO 27001 A.5.15 NIST 800-190 §4.4	Implemented	billing-app-01, operations-app-01, procurement-app-01, rotation-job-01, break-glass-01 — each with distinct API key and scoped grants.
Least privilege: identities granted only the privileges their function requires	ISO 27001 A.5.18 NIST 800-190 §4.4	Implemented	billing: read/execute on 5 vars. operations/procurement: read/execute on 1 var each. rotation-job: update only (no read) on billing db/password. Tiering derived from five-question framework.
Separation of duty: rotation identity cannot read the secret it rotates	ISO 27001 A.5.3	Implemented	rotation-job-01 has update (write) without read/execute on billing db/password. Can change but cannot retrieve.
Break-glass identity with cross-branch scope, post-use key rotation	ISO 27001 A.5.16 A.5.24	Implemented	break-glass-01 reads all app secrets. Key rotated after each use via conjur host rotate-api-key. Usage logged in audit trail.
Platform-native workload identity (no stored credential)	ISO 27001 A.5.16 A.8.24	Implemented	Phase 5: authn-jwt authenticator. A Kubernetes pod authenticates with its own service account JWT (issued by the cluster, mounted automatically, never stored). No API key in the pod spec, environment, or mounted secret. Validated by signature, issuer, and audience claims.
Administrative access requires named identity, not shared account	ISO 27001 A.5.17	Documented (gap)	Lab uses a single 'admin' identity. Production would assign named user identities per engineer, ideally federated via SSO/Entra ID.
MFA for vault administrative access	ISO 27001 A.8.5	Out of scope	Conjur OSS native authenticator does not support MFA. Production would use authn-oidc with an MFA-enforcing IdP.
Approval workflow for break-glass access	ISO 27001 A.5.24	Documented	Lab relies on key custody and post-use rotation. Production would gate access behind dual-control approval (PVWA / ITSM integration).

1.2 Secrets management and lifecycle

Control	Standard ref	Status	Implementation / gap
Secrets stored encrypted at rest in a dedicated vault	ISO 27001 A.8.24 NIST 800-190 §4.3	Implemented	All secrets stored in Conjur, encrypted in Postgres using the CONJUR_DATA_KEY symmetric key.
Secrets never hardcoded in application source	ISO 27001 A.8.24	Implemented	Three patterns demonstrated: direct API (runtime fetch), Summon (env var injection), Secretless (app never sees secret). API key moved from source to env var as hygiene step.
Automated credential rotation with no drift window	ISO 27001 A.5.17	Implemented	rotate_billing_password.sh: authenticates, fetches rotator credential from vault, generates random 24-char password, alters Postgres role, updates vault. Observed sub-second execution (no drift).

Control	Standard ref	Status	Implementation / gap
Manual rotation procedure documented and tested	ISO 27001 A.5.17	Implemented	Manual two-step rotation tested; drift window observed (~60s); all three app patterns confirmed to self-heal after vault resync.
Revocation effective immediately, per identity	ISO 27001 A.5.18	Implemented	!deny via policy update. All three patterns lost access immediately, no cached credentials survived. Confirmed per-fetch resolution across direct API, Summon, and Secretless.
Master key custody procedure	ISO 27001 A.8.24	Documented (gap)	Data key stored as a local file and shell variable. Production would use external KMS (Azure Key Vault, AWS KMS) or HSM for key material custody.
Scheduled automated rotation	ISO 27001 A.5.17	Documented	Cron entry documented (0 2,14 * * *) but not enabled due to WSL cron persistence limitations. On-demand script proven working.

1.3 Network and transport security

Control	Standard ref	Status	Implementation / gap
All vault API traffic encrypted in transit (TLS)	ISO 27001 A.8.24 CIS Docker 5.1	Implemented	nginx terminates TLS on 443. All client-to-vault calls over HTTPS. Self-signed certificate used.
Single externally reachable entry point (reverse proxy)	CIS Docker 5.1 NIST 800-190 §4.5	Implemented	Only nginx_proxy publishes a port for vault access. conjur_server and postgres_database are not directly reachable from the host (except via separately published ports for database admin).
Container network segmentation	CIS Docker 5.2 NIST 800-190 §4.5	Documented (gap)	All containers share one flat Docker network. Production would use separate frontend/backend networks, Kubernetes NetworkPolicies, or cloud security groups.
Mutual TLS between services	ISO 27001 A.8.24	Out of scope	Lab uses one-way TLS (server cert only). Production would add mTLS between vault, broker, and database, typically via a service mesh.
CA-issued certificates with defined rotation	ISO 27001 A.8.24	Out of scope	Self-signed certificate regenerated on stack restart; broke CLI and broker trust multiple times. Production would use internal/public CA-issued certs with scheduled rotation.
Broker back-side SSL to database	CIS Docker 5.1	Documented (gap)	sslmode=disable set in secretless.yml for lab. Production would enable TLS on Postgres and remove this override.

1.4 Audit and monitoring

Control	Standard ref	Status	Implementation / gap
All authentication attempts logged (success and failure)	ISO 27001 A.8.15 NIST 800-190 §4.6	Implemented	Conjur logs every authn attempt with origin IP, request ID, timestamp, and identity. Failed attempts (wrong key, expired token, YAML errors) also logged.
All policy changes logged with actor identity	ISO 27001 A.8.15	Implemented	policy load (POST) and policy update (PATCH) logged with myConjurAccount:user:admin, individual permission grants/removals logged separately. Verified in docker compose logs.
Audit logs shipped to external SIEM	ISO 27001 A.8.15 A.8.16	Documented	Logs remain in Docker container stdout. Production would forward to Microsoft Sentinel via syslog or the Conjur audit API.
Detection rules for	ISO 27001 A.8.16	Documented	Defined rules: break-glass-01 any authentication;

Control	Standard ref	Status	Implementation / gap
anomalous vault access			rotation-job-01 outside scheduled windows; repeated 401s from any identity; policy changes outside change windows. Not implemented (no SIEM connected).
Distinct error codes for auth failure vs permission denial	ISO 27001 A.8.15	Implemented (with caveat)	Conjur returns 404 (not 403) for permission denials to prevent resource enumeration. Documented as a deliberate design choice and a troubleshooting consideration.

1.5 Container and host hardening

Control	Standard ref	Status	Implementation / gap
Containers run as non-root where possible	CIS Docker 4.1	Partially implemented	Client container runs as 'cli' user. Conjur and Postgres run as their own application users. Not audited with a formal benchmark scan.
Container images scanned for vulnerabilities	CIS Docker 4.3 NIST 800-190 §4.1	Out of scope	No image scanning performed. Production would use Trivy, Grype, or Docker Scout.
Secretless broker plugin integrity verification	CIS Docker 4.3	Documented (gap)	Broker logs WARN: 'Plugin hashes were not provided — tampering will not be detectable.' Production would enable hash verification.
Bootstrap credential files restricted to owner-only	CIS Docker 3.22	Implemented	~/.rotation_env chmod 600; ~/.netrc chmod 600. Owner-only read/write on all credential files.

2. Application Onboarding Runbook

This runbook covers onboarding a new application identity into the vault, from initial scoping through to verified secret retrieval. It applies regardless of which delivery pattern (direct API, Summon, Secretless) is selected.

2.1 Pre-onboarding: scope the identity

Before writing any policy, answer all five questions for the application being onboarded:

#	Question	Expected answer format
1	What is this identity's real job?	One sentence in business language, e.g. 'processes customer card and bank transfer payments.'
2	What systems does that job require it to reach, and how does it authenticate to each?	Named list: system name + credential type (API key, certificate, token, connection string).
3	For each secret, does it need to look at it, use it, or change it?	Per-secret: read only, read+execute, or update. No 'probably needs access.'
4	Who owns this identity, and who is responsible for its lifecycle?	Named person or team, not 'IT.'
5	If this identity were compromised tomorrow, is the blast radius proportional to its job?	One sentence: 'damage would be limited to X' — if X exceeds the job scope, narrow the access.

Gate: Do not proceed to policy creation until all five questions are answered and documented.

2.2 Create the policy

Write a YAML policy file following this structure. The file creates the host identity, the variable slots, and the permission grants in one load:

```
- !policy
  id: <environment>              # e.g. prod
  body:
    - !policy
      id: <app-name>
      annotations:
        owner: <owner from Q4>
      body:
        - !variable <secret-path>   # one per secret from Q2
        - !host <app-identity>
        - !permit
          role: !host <app-identity>
          privileges: [ <from Q3> ] # read, execute, and/or update
          resource: !variable <secret-path>
```

If the application has multiple secrets with different access levels, use separate !permit blocks for each rather than a shared anchor. Load with:

```
conjur policy load -b root -f <filename>.yml
```

2.3 Set the secret values

For each variable created, set its value using `conjur variable set -i <path> -v '<value>'`. Confirm retrieval as admin with `conjur variable get -i <path>`. If using Summon or Secretless, all referenced variables must have values or the launch/connection will fail (all-or-nothing behaviour).

2.4 Configure the delivery mechanism

Pattern	Configuration required	Key files
Direct API	Application code imports Conjur SDK, authenticates as the host identity, calls .get() at runtime.	Application source, conjur-server.pem (cert)
Summon	Create secrets.yml mapping env-var names to vault paths. Set up .conjurrc and .netrc on the host with the identity's connection details and API key.	secrets.yml, ~/.conjurrc, ~/.netrc, conjur-server.pem
Secretless	Create secretless.yml defining the connector, listener, and credential mappings. Run the broker container with the identity's credentials as environment variables.	secretless.yml, conjur-server.pem, docker run command with CONJUR_* env vars

2.5 Verify and document

- Run the application and confirm it successfully retrieves the secret and performs its intended function (database write, API call, etc.).
- Check the Conjur audit log (docker compose logs conjur | grep <identity>) and confirm the authentication and retrieval events appear.
- Update the secret inventory (Section 4) with the new variables, owners, and rotation schedule.
- Update the break-glass policy if the new application's secrets should be accessible during emergencies.
- File a change record documenting the onboarding.

3. Credential Rotation Runbook

3.1 Automated rotation (preferred)

The automated rotation script (rotate_billing_password.sh) performs both the target-side change and the vault update as one operation. For billing's database password:

```
# Ensure the rotation API key is available
source ~/.rotation_env

# Run the rotation
./rotate_billing_password.sh

# Verify
tail -3 rotation.log   # expect: started, db changed, vault updated
python3 billing_app_secretless.py   # expect: successful row write
```

The script authenticates as rotation-job-01, fetches the rotator database credential from the vault, generates a 24-character random password, applies ALTER ROLE on the database, and writes the new value back to the vault. If any step fails, set -euo pipefail aborts the script before subsequent steps execute, preventing partial updates and drift.

3.2 Manual rotation (fallback)

If the automated script is unavailable, perform the two-step manual process. Be aware this creates a drift window during which applications will fail:

```
# Step 1: Change on the target
docker compose exec database psql -U postgres \
  -c "ALTER ROLE billing_app WITH PASSWORD '<new-password>';"

# Step 2: Update the vault (inside client container)
conjur variable set -i prod/billing-app/db/password -v '<new-password>'

# Step 3: Verify application recovery
python3 billing_app_secretless.py
```

Drift window: Between steps 1 and 2, all applications authenticating with the old password will fail. Minimise this window. In production, CyberArk CPM eliminates this by performing both steps atomically.

3.3 Rotating the bootstrap credentials

The rotation job's own API key and the host application identity keys should be rotated on a defined schedule. These are rotated via the Conjur CLI:

```
# Rotate a host identity's API key (run as admin from client container)
conjur host rotate-api-key -i prod/db-admin/rotation-job-01

# Update the stored copy wherever the old key was held
# For rotation-job-01: update ~/.rotation_env with the new key
# For billing-app-01: update ~/.netrc and recreate the Secretless broker
```

3.4 Post-rotation checklist

- Confirm the application works with the new credential (test by running the app, not by inspecting the value).
- Check the audit log for the rotation events (rotation-job-01 authenticating, variable update).
- Record the rotation date and next scheduled rotation in the secret inventory.
- If the rotation was unscheduled (emergency), file an incident record.

4. Key Custody Procedure

This section documents what key material exists, where it is stored, who is responsible for it, and how it is protected. In this lab environment, several credentials cannot be vaulted because they are the credentials used to reach the vault itself.

4.1 Key material inventory

Key material	Location	Protection	Custody owner	Rotation schedule
CONJUR_DATA_KEY (master encryption key)	File on host: ~/conjur-quickstart/data_key Exported as env var at stack start	chmod 600 Not backed up	Vault administrator	Never rotated in this lab. Production: external KMS/HSM, defined rotation per compliance framework.
Admin API key	File: ~/conjur-quickstart/admin_data Used inside client container	chmod 600 on file Session in .netrc inside client	Vault administrator	Rotate after any suspected exposure. Lab: not scheduled.
billing-app-01 API key	~/.netrc on host (for Summon) Broker env var (for Secretless)	chmod 600 on .netrc Env var in container memory	Application owner (billing)	Rotate on schedule and after any exposure.
rotation-job-01 API key	~/.rotation_env on host	chmod 600	PAM / platform team	Rotate after every known exposure. Scheduled rotation recommended.
break-glass-01 API key	Stored securely by Security Operations	Rotated after every use	Security Operations	Immediately after each use; not scheduled.
Postgres rotator password	Vaulted: prod/db-admin/rotator-password	Conjur encryption	Database team	Rotate on schedule; update both vault and database.

4.2 Credential chain analysis

The following chain describes the trust dependencies in the current lab. Each credential protects or gates access to the one below it:

- Level 0 (unprotected): CONJUR_DATA_KEY — protects all encrypted data in Postgres. Stored as a file and env var. Loss = permanent, unrecoverable data loss.
- Level 1 (unprotected in the API-key patterns): Host identity API keys (billing-app-01, rotation-job-01, break-glass-01) — stored in plaintext files (.netrc, .rotation_env, broker env). These are the bootstrap credentials used by the direct-API, Summon, and Secretless patterns.
- Level 1 eliminated (Phase 5): Kubernetes pod identity via authn-jwt — the service account token is issued by the cluster at runtime, mounted automatically, never handled by a person, and never stored as a static credential. This is the top rung of the chain and the concrete realisation of the platform-native identity model.
- Level 2 (vaulted): Application secrets (db/password, api/stripe-key, etc.) and the rotator database password — stored encrypted in Conjur, accessible only via Level 1 credentials or the Phase 5 platform identity.
- Level 3 (target systems): The actual database and external API accounts — protected by Level 2 credentials, which are managed and rotated via the vault.

Residual risk and its elimination: Level 1 credentials apply to the direct-API, Summon, and Secretless patterns. Phase 5 demonstrates the elimination of Level 1 for Kubernetes workloads: a pod authenticates with its own service account JWT via the authn-jwt authenticator, with no static key stored anywhere. Where platform-native identity is

unavailable, the compensating controls (file permissions, rotation, monitoring) reduce but do not remove the risk; production extends the same elimination to other workloads via authn-azure / Managed Identity and authn-iam.

5. Break-Glass Procedure

This procedure governs the use of the break-glass-01 emergency access identity. It has read/execute access to all application secrets across all three branches (billing, operations, procurement). Its use is exceptional and must be justified, logged, and followed by immediate key rotation.

5.1 When to invoke break-glass

- A production incident requires access to a credential and the normal application identity is non-functional (compromised, locked out, or access revoked during investigation).
- A security investigation requires verifying the current value of a secret across multiple application branches simultaneously.
- Normal administrative access (admin identity) is unavailable due to a credential loss or vault configuration issue.

Not for: Routine access, testing, password resets for convenience, or any action that can be performed through the normal identity or the admin account.

5.2 Execution steps

Step	Action	Command / detail
1	Obtain the break-glass API key from secure storage.	The key is held by Security Operations. Retrieve it from the secured location (in production: a physical safe, a separate vault, or a dual-control-gated store).
2	Authenticate as break-glass-01 and retrieve the required secret.	TOKEN=$(curl -s --cacert ./conjur-server.pem -d "<key>" "https://localhost:8443/authn/myConjurAccount/host%2Fprod%2Fbreak-glass%2Fbreak-glass-01/authenticate" \| base64 \| tr -d '\r\n') curl -s --cacert ./conjur-server.pem -H "Authorization: Token token=\"$TOKEN\"" "https://localhost:8443/secrets/myConjurAccount/variable/<path>"
3	Perform the emergency action.	Use the retrieved credential for the specific incident remediation task only.
4	Rotate the break-glass API key immediately.	conjur host rotate-api-key -i prod/break-glass/break-glass-01 (run as admin from client container)
5	Record the incident.	Document: who invoked it, when, which secrets were accessed, the justification, and the remediation action taken.
6	Review the audit trail.	docker compose logs conjur | grep break-glass Verify only the expected authentication and retrieval events appear.

5.3 Production enhancements (not implemented in lab)

- Dual-control approval: break-glass access gated behind two-person authorisation via PVWA or ITSM integration.
- Automatic time-boxing: access expires after a defined window (30 min, 2 hours) without manual revocation.
- Automatic post-use rotation: credential rotates itself when the session ends.
- Real-time SIEM alerting: any break-glass-01 authentication triggers an immediate page to the security team, not just a log entry.
- Session recording: all actions performed during the break-glass session are captured for forensic review.

6. Vulnerability and Benchmark Scan Methodology

A formal vulnerability scan of the container images and a CIS Docker Benchmark assessment of the host configuration were not performed during this lab build. This section documents the methodology that would be applied and the tools that would be used, for execution as a follow-up activity.

6.1 Container image scanning

Each container image in the stack should be scanned for known vulnerabilities (CVEs) before deployment. The following images are in scope:

Image	Scan tool	Notes
cyberark/conjur	Trivy or Grype	Primary vault image. Critical/High CVEs in Ruby or Rails dependencies are directly relevant to vault security.
postgres:15	Trivy or Grype	Backing store. CVEs in libpq or OpenSSL affect encryption at rest.
nginx:latest	Trivy or Grype	TLS termination point. CVEs in OpenSSL or nginx core are high priority.
cyberark/secretless-broker:latest	Trivy or Grype	Sits in the live data path. Go runtime and plugin CVEs are relevant.
cyberark/conjur-cli:9	Trivy or Grype	Administrative tool. Lower priority but should be scanned.

Example scan command (Trivy):

```
trivy image --severity CRITICAL,HIGH cyberark/conjur
trivy image --severity CRITICAL,HIGH postgres:15
trivy image --severity CRITICAL,HIGH nginx:latest
trivy image --severity CRITICAL,HIGH cyberark/secretless-broker:latest
```

6.2 CIS Docker Benchmark

The CIS Docker Benchmark (v1.6) provides a checklist of host and container configuration hardening recommendations. Docker Bench for Security is an automated tool that checks compliance:

```
docker run --rm --net host --pid host --userns host --cap-add audit_control \
  -e DOCKER_CONTENT_TRUST=$DOCKER_CONTENT_TRUST \
  -v /var/lib:/var/lib:ro -v /var/run/docker.sock:/var/run/docker.sock:ro \
  -v /etc:/etc:ro --label docker_bench_security \
  docker/docker-bench-security
```

The output categorises findings as PASS, WARN, NOTE, or INFO. Priority areas for this lab environment:

- Section 4 (Container images and build): image provenance, running as non-root, read-only root filesystems.
- Section 5 (Container runtime): network segmentation, published ports, privileged mode, resource limits.
- Section 7 (Docker Swarm / orchestration): not applicable to this lab (plain Docker Compose).

6.3 Scan report template

When executed, the scan report should include: date of scan, image versions scanned, tool and version used, count of Critical/High/Medium/Low findings per image, remediation actions taken or accepted, and a sign-off by the responsible engineer. Results should be retained alongside this governance documentation.

7. Secret Inventory

Current inventory of all secrets stored in the vault, with ownership, purpose, and rotation status.

Variable path	Owner	Purpose	Last rotated	Rotation method
prod/billing-app/db/password	Billing (unset)	Postgres billing_app role password	Last automated rotation run	Automated (rotate_billing_password.sh)
prod/billing-app/api/stripe-key	Billing (unset)	Stripe API key (fake lab value)	Never	Manual (Stripe dashboard + variable set)
prod/billing-app/db/host	Billing (unset)	Database hostname (internal: 'database')	N/A (static)	N/A
prod/billing-app/db/port	Billing (unset)	Database port (internal: '5432')	N/A (static)	N/A
prod/billing-app/db/username	Billing (unset)	Database username ('billing_app')	N/A (static)	N/A
prod/operations-app/powerbi-connection-string	Khuli Shaka	PowerBI connection string (fake lab value)	Never	Manual
prod/procurement-app/vendor-api-key	Alan Simba	Vendor portal API key (fake lab value)	Never	Manual
prod/db-admin/rotator-password	Database Team	Postgres rotator role password	Never	Manual (ALTER ROLE + variable set)

8. Access Matrix

This matrix shows which identity can access which secret, and with what privilege. Empty cells indicate no grant.

Secret	billing-app-01	operations-app-01	procurement-app-01	rotation-job-01	break-glass-01	admin
billing db/password	read, execute			update	read, execute	full
billing api/stripe-key	read, execute				read, execute	full
billing db/host	read, execute					full
billing db/port	read, execute					full
billing db/username	read, execute					full
operations powerbi-connection-string		read, execute			read, execute	full
procurement vendor-api-key			read, execute		read, execute	full
db-admin rotator-password				read, execute		full

Not shown above (to keep the matrix readable): the Phase 5 JWT identity `prod/k8s-apps/system:serviceaccount:billing-app:billing-sa` has read/execute on billing db/password. Unlike every identity in the matrix, it authenticates with no stored API key — its Kubernetes service account token is its credential.

9. Consolidated Production Considerations

This section consolidates all production-readiness gaps identified throughout the lab build, in one place. Each item names the lab limitation, the production control that addresses it, and a reference to where it was encountered.

9.1 Infrastructure and deployment

- Single-instance deployment (one Conjur, one Postgres, one nginx). Production: clustering/HA for Conjur, HA Postgres (managed database service with replication), load-balanced proxy tier, and a separate DR vault.
- Conjur deployed via Docker Compose on a laptop. Production: Kubernetes-based Conjur Leader/Follower architecture, with Followers distributed close to consuming applications.
- Secretless broker not part of docker-compose.yml; must be started/recreated separately and does not survive stack restarts. Production: broker runs as a Kubernetes sidecar in the same pod as the application, managed by the pod lifecycle.

9.2 Certificate and TLS

- Self-signed certificate regenerates on stack restart; broke CLI and broker trust multiple times. This is the single most impactful operational fragility in the lab. Production: CA-issued certificates (internal or public CA) with defined rotation schedule. Clients validate against the CA root, not a pinned leaf, so leaf rotation is transparent.
- Broker back-side connection to database uses sslmode=disable. Production: TLS enabled on Postgres; mTLS between all components via service mesh (Istio, Linkerd).

9.3 Identity and authentication

- The direct-API, Summon, and Secretless patterns authenticate via native authn (API key for token), with bootstrap keys in plaintext files. Phase 5 demonstrates the alternative: the authn-jwt authenticator, where a Kubernetes pod authenticates with its own service account token and no static key. Production extends the same model to non-Kubernetes workloads via authn-azure (Managed Identity) and authn-iam, and to human users via authn-oidc (SSO/Entra ID).
- Single shared 'admin' identity used for all administrative operations. Production: named user identities per engineer, federated via SSO, with MFA enforced by the IdP.

9.4 Credential lifecycle

- Manual/scripted rotation replaces CyberArk CPM. Production: CPM performs target-side password change and vault update as one coordinated, scheduled, fully automated operation using a managed reconciliation account.
- Cron scheduling not enabled due to WSL persistence limitations. Production: scheduled rotation via Kubernetes CronJob, systemd timer, or CPM's built-in scheduler.

9.5 Break-glass

- Break-glass access is unrestricted once the API key is known; no approval gate, no time-boxing, no automatic post-use rotation. Production: dual-control approval (PVWA/ITSM), automatic session time-boxing, automatic key rotation on session end, real-time SIEM alerting on any break-glass authentication, and full session recording.

9.6 Monitoring and audit

- Audit logs remain in Docker container stdout. Production: forward to SIEM (Microsoft Sentinel) via syslog or Conjur audit API, with defined retention (90 days to 1 year per compliance framework).

- Detection rules defined but not implemented: break-glass any-auth, rotation-job off-schedule, repeated 401s, policy changes outside change windows. Production: implemented as Sentinel analytics rules with automated alerting.
- Secretless broker logs are the only source of credential-failure diagnostics for brokered applications. Production: broker logs must be shipped and indexed alongside application and vault logs for effective incident investigation.

9.7 Container hardening

- No formal CIS Docker Benchmark scan or container image vulnerability scan performed. Production: Trivy/Grype image scanning in CI/CD pipeline; Docker Bench for Security against the host; findings remediated or accepted with documented rationale.
- Secretless broker plugin hash verification not enabled (WARN logged). Production: enable to detect plugin tampering.

End of governance documentation.